

CPU Architecture and Memory Hierarchy: Cores, Cache, RAM, and Data Access

Computer architecture describes how the main hardware components of a computer system are designed, organized, and connected.

For AI infrastructure, the most important processing components are the **CPU and GPU**. The CPU manages the operating system, applications, control logic, data preparation, and system coordination. The GPU performs large numbers of mathematical operations in parallel, making it particularly effective for AI training, inference, scientific computing, and data processing.

In an AI infrastructure environment, CPUs and GPUs normally work together rather than replacing one another. The CPU acts as the host and system coordinator, while the GPU acts as an accelerator for computationally intensive parallel workloads.

1. What is a CPU?

A **Central Processing Unit**, or CPU, is a general-purpose processor responsible for executing instructions, running the operating system, controlling applications, managing system resources, and coordinating other hardware components.

The CPU is often described as the “brain” of the computer because it interprets program instructions and determines what operations must be performed. It is designed to handle a wide variety of tasks, including calculations, decision-making, input/output operations, memory management, process scheduling, and application control.

A CPU typically contains a relatively small number of powerful processing cores. Each core is optimized to execute complex instruction sequences quickly and efficiently. This makes CPUs especially effective for workloads that involve sequential processing, complex branching, irregular logic, and tasks that cannot easily be divided into thousands of identical operations.

Main Responsibilities of a CPU

A CPU commonly performs the following tasks:

- Runs the operating system.
- Executes application instructions.
- Manages files, processes, and system services.

- Coordinates memory access.
- Performs data preprocessing.
- Manages input/output devices.
- Schedules work for GPUs and other accelerators.
- Handles complex decision-making and branching.
- Controls communication among hardware components.

In an AI server, the CPU may read training data from storage, prepare batches, initiate GPU operations, manage network connections, and monitor the overall training process.

Real-World Example

Suppose an AI training application uses thousands of images.

The CPU may:

1. Locate the images in storage.
2. Read the files.
3. Decode and resize the images.
4. Arrange them into batches.
5. Transfer the batches to GPU memory.
6. Launch the GPU computation.
7. Manage logging and checkpoint operations.

The GPU then performs the neural-network calculations on those image batches.

2. CPU Architecture

CPU architecture refers to the internal design and organization of a CPU, including its cores, registers, execution units, caches, control unit, instruction set, memory interface, and communication pathways.

The architecture determines how the CPU receives instructions, processes data, accesses memory, and communicates with the rest of the system.

Major Components of CPU Architecture

Control Unit

The control unit directs CPU operations. It fetches instructions from memory, decodes them, and sends control signals to the appropriate execution units.

It determines:

- Which instruction should run.
- Which data should be used.
- Which execution unit should perform the work.
- Where the result should be stored.

Arithmetic Logic Unit

The **Arithmetic Logic Unit**, or ALU, performs integer arithmetic and logical operations.

Examples include:

- Addition
- Subtraction
- Comparisons
- Boolean operations
- Bit shifting
- Address calculations

A modern CPU core may contain multiple execution units rather than a single ALU.

Floating-Point Unit

The **Floating-Point Unit**, or FPU, performs calculations involving decimal or floating-point numbers.

Floating-point operations are important for:

- Scientific computing
- Simulations
- Statistics

- Graphics
- Machine learning
- Engineering calculations

However, CPUs contain far fewer floating-point execution units than GPUs, which is one reason GPUs achieve much higher throughput on large parallel mathematical workloads.

Registers

Registers are extremely small and extremely fast storage locations inside the CPU core.

They hold:

- Current instructions
- Memory addresses
- Intermediate values
- Function arguments
- Calculation results

Registers are faster than CPU cache and system memory, but their capacity is very limited.

Instruction Decoder

The instruction decoder translates program instructions into internal operations that the processor can execute.

For example, it may decode an instruction telling the CPU to:

- Add two numbers.
- Load data from memory.
- Compare two values.
- Jump to another instruction.

Store a result.

Branch Prediction

Branch prediction is a hardware technique used by modern central processing units (CPUs) to guess the outcome of a conditional decision (such as an if statement or a loop) before it is actually calculated, keeping the processor pipeline full and running fast

Programs frequently contain decision statements such as if, else, loops, and function calls. The CPU attempts to predict which instruction path will be needed before the result of the decision is known.

Correct branch prediction keeps the processing pipeline busy. Incorrect prediction causes the CPU to discard some work and restart execution from the correct path.

CPUs devote considerable hardware to branch prediction because they are designed for complex and irregular workloads.

Execution Pipeline

A CPU processes instructions through stages such as:

1. Fetch
2. Decode
3. Issue
4. Execute
5. Access memory
6. Write back the result

Modern CPUs overlap these stages so that several instructions can be processed simultaneously at different points in the pipeline.

Instruction Set Architecture

The **Instruction Set Architecture**, or ISA, defines the instructions, registers, data types, and memory-addressing methods visible to software.

Examples include:

- x86-64
- ARM
- RISC-V

Software must be compiled or interpreted for the instruction set supported by the CPU.

CPU architecture generally dedicates a significant portion of its transistors to:

- Complex control logic
- Large caches
- Branch prediction
- Out-of-order execution
- Low-latency instruction processing

This design is excellent for general-purpose computing but leaves comparatively less chip area for large numbers of arithmetic units.

3. CPU Cores

A CPU core is an independent processing unit within a CPU that can fetch, decode, and execute instructions.

A processor with multiple cores can execute multiple instruction streams at approximately the same time. For example:

- A 4-core CPU can execute several tasks concurrently.
- A 16-core CPU can handle more simultaneous work.
- A server CPU may contain dozens or more cores.

Each core usually contains its own registers, execution units, pipelines, and private cache resources. Some higher-level caches may be shared among multiple cores.

Physical Cores and Logical Cores

Physical cores are the actual hardware processing units on a computer's CPU chip, while logical cores are virtual pathways (threads) created using technologies like Hyper-Threading or SMT. One physical core can run multiple logical cores by sharing its processing power and filling idle moments.

- **Physical Core:** An actual hardware processing unit inside a CPU that executes instructions and performs computations.
- **Logical Core:** A virtual processing unit presented to the operating system that allows a physical core to handle more than one thread of work.

Example: A CPU with **8 physical cores and 16 logical cores** has 8 real processing cores, with each core able to handle 2 threads.

A **physical core** is like an actual **worker** in the office.

A **logical core** is like giving that worker the ability to handle **two tasks at once** when possible.

For example:

CPU with 4 physical cores:

Worker 1 → Physical Core 1

Worker 2 → Physical Core 2

Worker 3 → Physical Core 3

Worker 4 → Physical Core 4

If each core supports 2 threads:

Physical Core 1 → Logical Core 1 + Logical Core 2

Physical Core 2 → Logical Core 3 + Logical Core 4

Physical Core 3 → Logical Core 5 + Logical Core 6

Physical Core 4 → Logical Core 7 + Logical Core 8

So the CPU is:

4 physical cores / 8 logical cores

The important part: **8 logical cores does NOT mean you have 8 real cores.** You still have 4 real physical cores. The CPU is simply better at keeping those 4 cores busy.

A simple way to remember it:

Physical core = actual hardware worker
Logical core = task slot available to that worker

So when your computer says:

Cores: 8

Logical processors: 16

it means **8 actual CPU cores, each capable of working on 2 instruction threads.**

CPU Core Strengths

CPU cores are optimized for:

- Strong single-thread performance
- Complex instruction execution

- Irregular control flow
- Branch-heavy applications
- Operating-system tasks
- Database transactions
- Application coordination
- Serial computation

A CPU core is much more complex than a typical GPU arithmetic unit.

CPU Cores in AI Infrastructure

CPU cores support AI workloads by handling:

- Dataset preprocessing
- Data decompression
- Data augmentation
- Application logic
- Container and operating-system services
- GPU kernel launching
- Network and storage coordination
- Workload scheduling
- Training-process management

Insufficient CPU resources can cause GPU underutilization if the CPU cannot prepare or deliver data quickly enough.

4. Memory hierarchy

Memory hierarchy is the way a computer organizes different types of memory and storage based on **speed, size, cost, and distance from the CPU**.

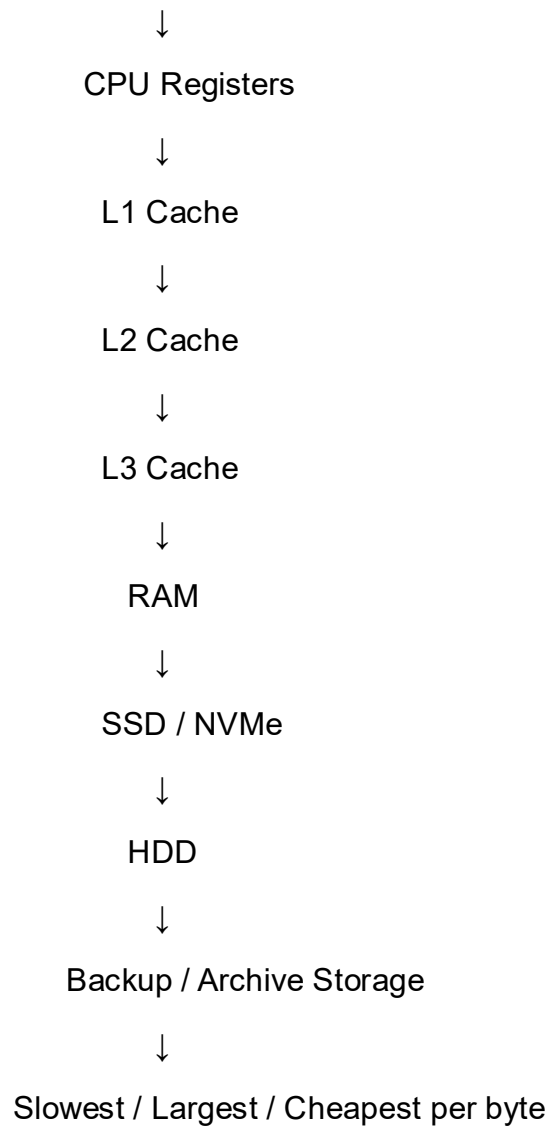
The basic principle is simple:

The closer memory is to the CPU, the faster and smaller it is.

The farther it is from the CPU, the slower and larger it is.

A typical hierarchy looks like this:

Fastest / Smallest / Most Expensive per byte



Why do computers need a memory hierarchy?

Ideally, we would want memory that is:

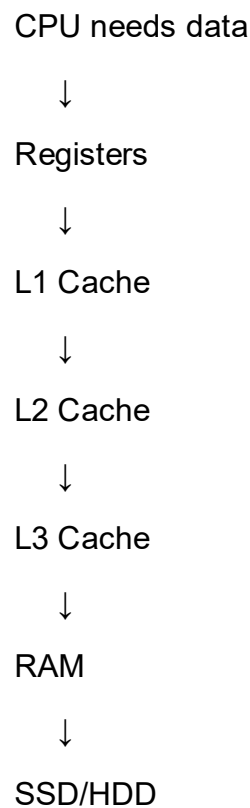
- extremely fast,
- extremely large,
- very cheap,
- and located directly next to the CPU.

In practice, you cannot optimize all of those properties at the same time.

Very fast memory is expensive and consumes more chip area. Large-capacity memory is cheaper, but slower. Because of that, computers use several layers.

The CPU first checks the fastest memory. If the data is not there, it moves to the next level.

Conceptually:



The goal is to keep the most frequently used data as close to the CPU as possible.

CPU Registers

Registers are the fastest storage locations in the computer.

They are located directly inside the CPU core and hold values that the processor is actively working with.

Registers are very small, extremely fast storage locations inside the CPU that hold instructions, addresses, and data currently being processed.

L1 Cache

L1 cache is the first level of CPU cache.

It is extremely fast and located very close to the CPU execution units.

Modern CPUs typically have separate:

- **L1 Instruction Cache**
- **L1 Data Cache**

The instruction cache stores instructions the processor may execute.

The data cache stores values the processor may use.

L1 cache is the smallest and fastest CPU cache level and stores the data and instructions most immediately needed by a CPU core.

Why L1 is small

Making cache larger usually increases complexity and access latency. Therefore, L1 is intentionally kept relatively small so that it can remain extremely fast.

L2 Cache

L2 cache is the next level.

It is:

- larger than L1,
- slower than L1,
- but still much faster than RAM.

If the CPU does not find data in L1, it checks L2.

L2 cache is a larger but somewhat slower cache than L1 and serves as the next level of fast memory before the processor accesses higher cache levels or RAM.

L3 Cache

L3 cache is normally larger than L1 and L2.

It is also generally slower.

A common difference is that **L3 is shared between multiple CPU cores.**

Example:

Core 1 → L1 → L2 —┐

Core 2 → L1 → L2 —┘

Core 3 → L1 → L2 —+—→ L3 Cache

Core 4 → L1 → L2 —┘

L3 acts as another layer before the CPU must access RAM.

When L3 is the final CPU cache before RAM, it is often called the: **Last-Level Cache (LLC)**

RAM — Main Memory

RAM is the computer's main working memory that stores active programs and data. It provides much larger capacity than CPU cache but has significantly higher access latency.

RAM stores the programs and data that are currently active.

For example, when you open:

- a web browser,
- a database,
- a virtual machine,
- a Python program,

their active data is generally placed into RAM.

SSD / NVMe Storage

Below RAM is persistent storage.

Examples include:

- SATA SSD
- NVMe SSD

An SSD stores information even when the computer is powered off.

This is a major difference from RAM.

RAM is generally **volatile**:

Power OFF

↓

RAM contents lost

SSD is **non-volatile**:

Power OFF



Data remains

Applications, operating systems, databases, files, and virtual-machine images are usually stored on persistent storage.

When needed, portions are loaded into RAM.

SSD



RAM



Cache



CPU

NVMe SSDs are much faster than older SATA SSDs, but they are still significantly slower than RAM.

HDD

A hard disk drive is another form of persistent storage.

It uses spinning magnetic disks and mechanical read/write heads.

Compared with SSDs, HDDs are generally:

- slower,
- cheaper per gigabyte,
- available in large capacities.

Why not make the entire RAM as fast as L1 cache?

This is an important interview follow-up.

Suppose a server has:

512 GB RAM

You might ask:

Why not build 512 GB of L1-speed memory?

Because very fast SRAM-style cache memory requires substantially more transistor area and cost than DRAM.

CPU cache is typically built using **SRAM**.

System RAM is typically built using **DRAM**.

SRAM is:

- faster,
- more expensive,
- less dense.

DRAM is:

- slower,
- cheaper,
- denser.

Therefore:

SRAM → CPU cache

DRAM → Main memory

Using SRAM for hundreds of gigabytes of system memory would be impractical in normal systems.

Memory hierarchy in servers

In modern servers, the hierarchy becomes even more important.

A simplified server could look like:

CPU Core

↓

Registers

↓

L1

↓

L2

↓

L3 / LLC



Local NUMA RAM



Remote NUMA RAM



NVMe SSD



Network Storage

Notice an additional concept:

NUMA — Non-Uniform Memory Access

In multi-socket systems, memory attached to the local CPU socket may be faster to access than RAM attached to another CPU socket.

Bandwidth vs Latency

Another important distinction:

Latency = How long one access takes.

Bandwidth = How much data can be transferred per second.

For example:

Latency:

"How quickly can the first byte arrive?"

Bandwidth:

"How many GB can I transfer every second?"

Memory systems care about both.

A CPU cache has very low latency and high bandwidth.

RAM has higher latency but much greater total capacity.

Storage has substantially higher latency and lower bandwidth than main memory.

What is memory hierarchy?

Memory hierarchy is the organization of computer memory into multiple levels based on speed, latency, capacity, and cost. The hierarchy typically starts with CPU registers, followed by L1, L2, and L3 cache, then RAM, and finally persistent storage such as SSDs and HDDs. Memory closer to the CPU is faster and has lower latency but is smaller and more expensive per byte. Memory farther from the CPU is slower but provides much larger capacity at a lower cost. The purpose of the hierarchy is to keep frequently accessed data close to the processor while still providing large overall storage capacity.

What is the typical order?"

**Registers → L1 Cache → L2 Cache → L3 Cache → RAM → SSD/NVMe → HDD
→ archival storage**